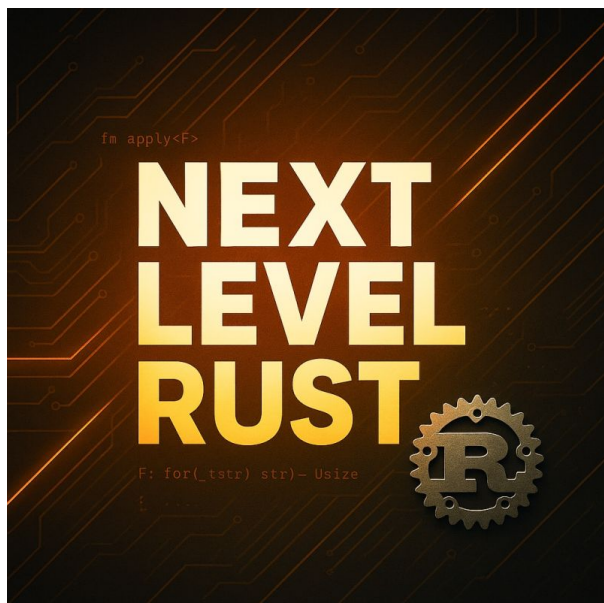




Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

Advanced cargo usage

Monorepo Strategies with Cargo

A monorepo is a single repository that contains multiple related projects or crates. In Rust, Cargo provides built-in support for managing monorepos through workspaces, shared configurations, and dependency resolution. This chapter explores effective strategies for organizing and maintaining a monorepo using only Cargo, without relying on external tools.

Why Use a Monorepo?

Monorepos are useful when you have multiple crates that evolve together—such as a library and its ecosystem of binaries, plugins, or examples. Benefits include:

- Unified versioning and release cycles
- Simplified cross-crate refactoring
- Shared tooling and configuration
- Atomic commits affecting multiple crates

Cargo's workspace system is ideal for implementing monorepos in Rust.

Flat vs. Nested Structure

There are two common ways to organize a monorepo with Cargo.

Flat Structure

All crates are placed in subdirectories at the same level:

```
my-monorepo/
├── Cargo.toml           (workspace root)
├── core/
│   ├── src/lib.rs
│   └── Cargo.toml
├── cli/
│   ├── src/main.rs
│   └── Cargo.toml
├── web/
│   ├── src/main.rs
│   └── Cargo.toml
└── tests/
    ├── src/main.rs
    └── Cargo.toml
```

Root `Cargo.toml`:

```
[workspace]
members = [
    "core",
    "cli",
    "web",
    "tests",
]
```

This is the most common and straightforward layout.

Nested Structure

Crates are grouped by purpose or domain:

```
my-monorepo/
├── Cargo.toml
├── libraries/
│   └── data-model/
│       ├── src/lib.rs
│       └── Cargo.toml
├── services/
│   ├── api-server/
│   │   ├── src/main.rs
│   │   └── Cargo.toml
│   └── worker/
│       ├── src/main.rs
│       └── Cargo.toml
```

```
└─ tools/
  └─ migrate/
    └─ src/main.rs
    └─ Cargo.toml
```

Root `Cargo.toml` uses glob-like patterns (Cargo supports this since 1.21):

```
[workspace]
members = [
  "libraries/*",
  "services/*",
  "tools/*",
]
```

This structure improves organization for large projects.

Shared Dependencies with Workspace Dependencies

To avoid version mismatches, define common dependencies in the workspace root.

Root `Cargo.toml`:

```
[workspace]
members = ["libraries/*", "services/*", "tools/*"]

[workspace.dependencies]
serde = { version = "1.0", features = ["derive"] }
tokio = { version = "1.0", features = ["full"] }
log = "0.4"
anyhow = "1.0"
```

Now, any crate can use these without repeating versions:

In `services/api-server/Cargo.toml`:

```
[dependencies]
tokio = { workspace = true }
serde = { workspace = true }
anyhow = { workspace = true }
data-model = { path = "../../libraries/data-model" }
```

This ensures consistency and reduces maintenance overhead.

Version Synchronization

In a monorepo, you may want all crates to share the same version number, especially if they are released together.

Cargo does not support global versioning directly, but you can manage this manually or use conventions.

For example, update versions in bulk:

```
# Update core
cd libraries/data-model && cargo set-version 0.5.0
# Update services
cd ../../services/api-server && cargo set-version 0.5.0
# Repeat for others
```

Note: `cargo set-version` requires the `cargo-edit` crate. If you prefer to avoid external tools, edit `Cargo.toml` files manually.

Alternatively, accept independent versioning within the monorepo. This is acceptable if crates have different release cycles.

Building and Testing Efficiently

From the root directory, build all crates:

```
cargo build
```

Build only a specific service:

```
cargo build -p api-server
```

Run tests across the entire monorepo:

```
cargo test
```

Or test only the core library:

```
cargo test -p data-model
```

Since all crates share a target directory and `Cargo.lock`, compilation is optimized and dependencies are deduplicated.

Publishing from a Monorepo

Not all crates in a monorepo need to be public. You can publish only specific members.

Before publishing, ensure the crate has:

- A valid license
- Description and repository in `Cargo.toml`
- Proper documentation

Publish one crate at a time:

```
cd services/api-server
cargo publish
```

Cargo will respect the workspace but publish only the current crate.

To prevent accidental publishing, add this to internal crates:

```
[package]
# ...
publish = false
```

This disables publishing for tools or private components.

Handling Examples and Integration Tests

Use the `examples/` directory inside a library crate to share runnable examples across the monorepo.

For instance, in `libraries/data-model/examples/demo.rs`:

```
use data_model::User;

fn main() {
    let user = User::new("Alice");
    println!("{:?}", user);
}
```

Run it with:

```
cargo run --example demo -p data-model
```

You can also write integration tests in one crate that depend on another, ensuring compatibility.

Avoiding Cyclic Dependencies

Monorepos make it easy to introduce cycles. For example, `api-server` depending on `worker`, and `worker` depending back on `api-server`.

Cargo will detect and reject such cycles. To prevent them:

- Define clear layering (e.g., libraries → services → tools)
 - Use workspace dependencies only when necessary
 - Keep high-level binaries at the edges
-

Summary

Monorepos in Rust are powerful when structured around Cargo workspaces. By using a clear directory layout, shared dependencies, and consistent configuration, you can manage many crates efficiently.

Key strategies include:

- Organizing crates with flat or nested member lists
- Using `[workspace.dependencies]` for version alignment
- Building and testing selectively with `-p`
- Publishing only public crates
- Preventing accidental publishing with `publish = false`

All of this is possible using only Cargo and the standard Rust toolchain. No external monorepo tools are required.

With thoughtful design, a Cargo-based monorepo becomes a scalable, maintainable foundation for complex Rust projects.